Tidy Data & Data Wrangling for Viz

Workshop 2 — Module 9

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

Module Roadmap

1. Tidy data: Wickham's three rules
2. Wide vs long format: `pivot_longer()` / `melt()`
3. Six core dplyr verbs with pandas equivalents
4. The wrangle-to-viz pipeline
5. Common data problems and their R/Python fixes
6. Joining data frames for multi-source visualizations

Learning Outcome
You will reshape any dataset into the tidy format that ggplot2 and seaborn require, and chain a complete wrangle-to-plot pipeline in both R and Python.

Tidy Data

Wickham's Three Rules

Tidy Data: Three Rules (Wickham, 2014)

1 **Every column is a variable**

name, date, revenue, type

2 **Every row is an observation**

One measurement per row

3 **Every cell is a single value**

No lists, no merged cells

Pro-Tip

ggplot2 requires tidy data: each aesthetic mapping (`aes(x=, y=, color=)`) expects a *column name*. If a variable is spread across multiple columns (wide format), you must reshape before plotting.

Wide vs Long Format

Wide → Long: The Reshape That Makes ggplot2 Work

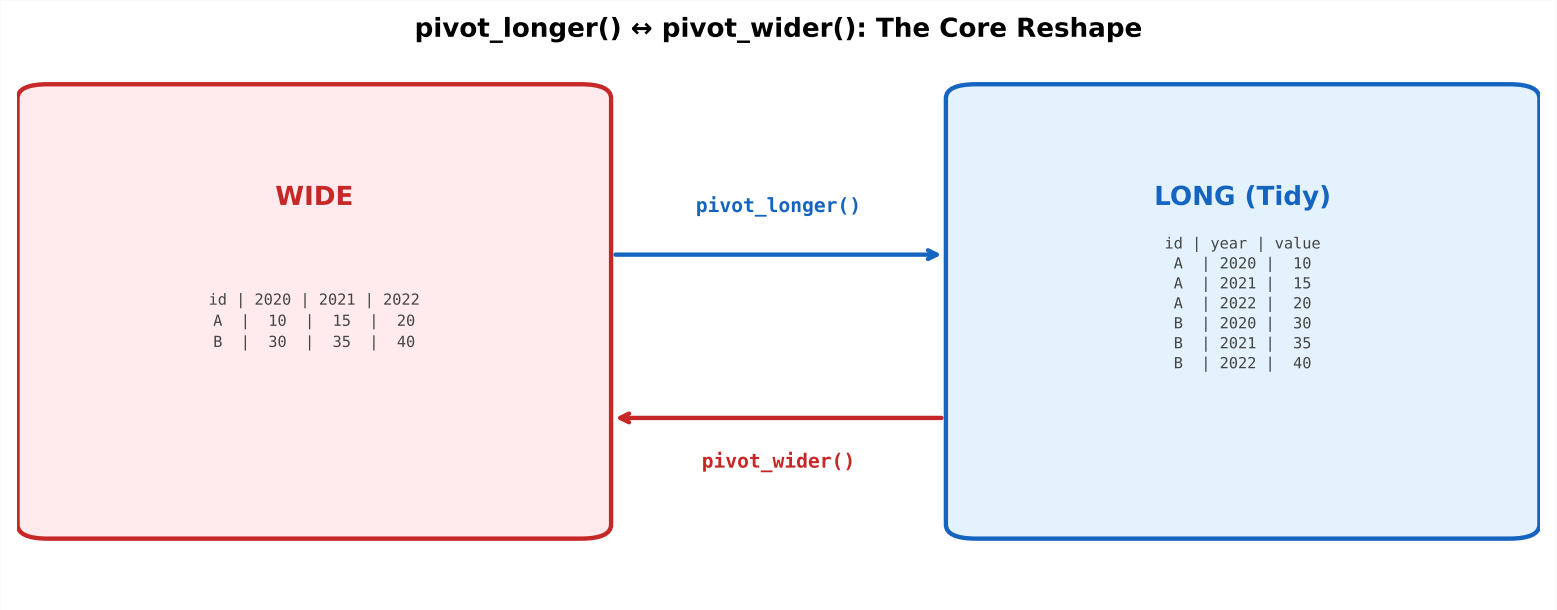
WIDE Format
(human-readable, NOT tidy)
Years as columns

Country	2020	2021	2022
Albania	2.9	7.5	4.8
Greece	-9.0	8.4	5.9
Italy	-9.0	6.7	3.7

LONG (Tidy) Format
(ggplot-ready, one obs per row)
pivot_longer()

Country	Year	GDP_Growth
Albania	2020	2.9
Albania	2021	7.5
Albania	2022	4.8
Greece	2020	-9.0
Greece	2021	8.4
Greece	2022	5.9

pivot_longer() ↔ pivot_wider()



Reshaping in R vs Python

R (tidyr)

```r
# Wide -> Long
df_long <- df_wide |>
  pivot_longer(
    cols = `2020`:`2022`,
    names_to = "year",
    values_to = "gdp_growth"
  )

# Long -> Wide
df_wide <- df_long |>
  pivot_wider(
    names_from = year,
    values_from = gdp_growth
  )
```

Python (pandas)

```python
# Wide -> Long
df_long = df_wide.melt(
    id_vars=["Country"],
    var_name="year",
    value_name="gdp_growth")

# Long -> Wide
df_wide = df_long.pivot_table(
    index="Country",
    columns="year",
    values="gdp_growth"
).reset_index()
```

Data Insight

Rule: if you need `aes(color = year)` but year is spread across columns, you must `pivot_longer()` first. The test: "Can I name a single column for this aesthetic?"

Six Core Verbs

dplyr Verbs with pandas Equivalents

Six Core dplyr Verbs (with pandas Equivalents)

`filter()`

Keep rows matching condition

```
R: df |> filter(Rating > 4)  →  pandas: .query()
```

`select()`

Keep/drop columns

```
R: df |> select(Rating, Type)  →  pandas: .[['Rating','Type']]
```

`mutate()`

Add/transform columns

```
R: df |> mutate(log_rev = log10(Reviews))  →  pandas: .assign()
```

`arrange()`

Sort rows

```
R: df |> arrange(desc(Rating))  →  pandas: .sort_values()
```

`group_by()`

Set grouping variable

```
R: df |> group_by(Category)  →  pandas: .groupby()
```

`summarise()`

Compute aggregates

```
R: df |> summarise(m = mean(Rating))  →  pandas: .agg()
```

The Six Verbs in R

```r
library(tidyverse)

apps <- read_csv("googleplaystore.csv") |>
  # 1. filter: keep rows
  filter(Type %in% c("Free", "Paid"), !is.na(Rating)) |>
  # 2. select: keep columns
  select(Category, Rating, Reviews, Type, `Content Rating`) |>
  # 3. mutate: transform columns
  mutate(Reviews = as.numeric(Reviews),
         log_reviews = log10(Reviews + 1)) |>
  # 4. arrange: sort
  arrange(desc(Rating))

# 5-6. group_by + summarise: aggregate
summary <- apps |>
  group_by(Category) |>
  summarise(
    n = n(),
    mean_rating = mean(Rating),
    se = sd(Rating) / sqrt(n()),
    .groups = "drop"
  ) |>
  slice_max(n, n = 8) |>
  mutate(Category = fct_reorder(Category, mean_rating))
```

The Six Verbs in Python

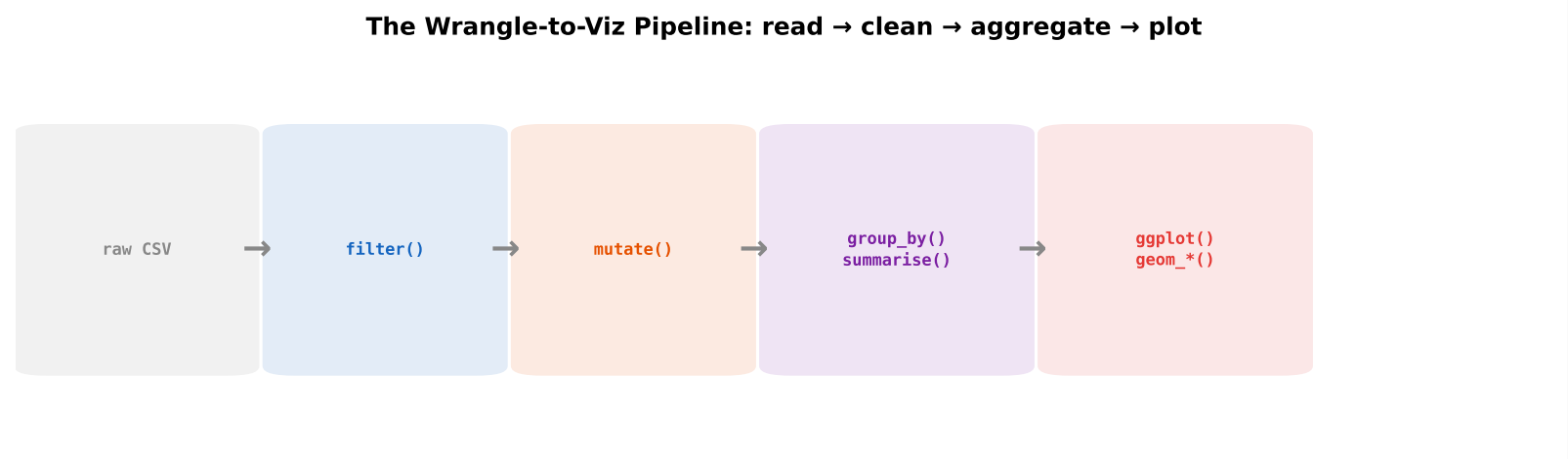
```python
import pandas as pd
import numpy as np

apps = (pd.read_csv("googleplaystore.csv")
    # 1. filter (query)
    .query("Type in ['Free', 'Paid']")
    .dropna(subset=["Rating"])
    # 2. select
    [["Category", "Rating", "Reviews", "Type", "Content Rating"]]
    # 3. mutate (assign)
    .assign(
        Reviews=lambda d: pd.to_numeric(d["Reviews"], errors="coerce
            "),
        log_reviews=lambda d: np.log10(d["Reviews"] + 1))
    # 4. arrange (sort_values)
    .sort_values("Rating", ascending=False))

# 5-6. group_by + summarise (groupby + agg)
summary = (apps
    .groupby("Category")
    .agg(n=("Rating", "count"),
         mean_rating=("Rating", "mean"),
         se=("Rating", lambda x: x.std() / np.sqrt(len(x))))
    .reset_index()
    .nlargest(8, "n")
    .sort_values("mean_rating"))
```

The Wrangle-to-Viz Pipeline

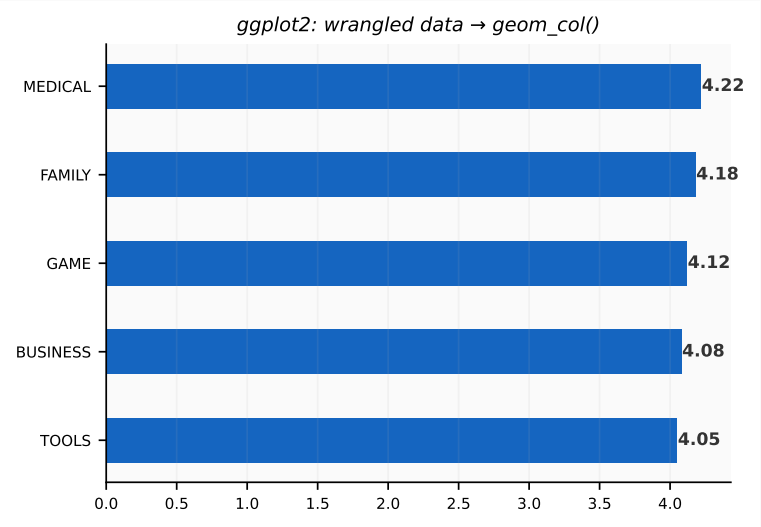
read $\rightarrow$ clean $\rightarrow$ aggregate $\rightarrow$ plot



Complete Pipeline: R

```r
read_csv("googleplaystore.csv") |>
  # WRANGLE
  filter(Type %in% c("Free","Paid")
      ,
         !is.na(Rating)) |>
  group_by(Category) |>
  summarise(
    n = n(),
    m = mean(Rating),
    se = sd(Rating)/sqrt(n()),
    .groups = "drop") |>
  slice_max(n, n = 8) |>
  mutate(Category =
    fct_reorder(Category, m)) |>

  # PLOT
  ggplot(aes(x = Category,
             y = m)) +
  geom_col(fill = "#1565C0",
           width = 0.5, alpha =
               0.6) +
  geom_errorbar(aes(
    ymin = m - se, ymax = m + se),
    width = 0.2) +
  coord_flip() +
  theme_minimal() +
```

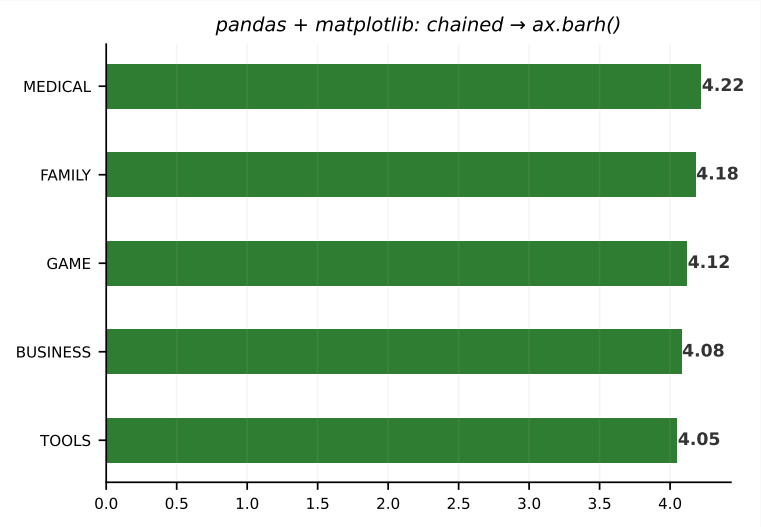


The entire pipeline — from CSV to publication chart — in one continuous chain. The pipe |> connects wrangling verbs; the + connects ggplot2 layers.

Complete Pipeline: Python

```python
summary = (pd.read_csv(
    "googleplaystore.csv")
  # WRANGLE
  .query("Type in ['Free','Paid']")
  .dropna(subset=["Rating"])
  .groupby("Category")
  .agg(n=("Rating","count"),
       m=("Rating","mean"),
       se=("Rating",
           lambda x: x.std()/
           np.sqrt(len(x))))
  .reset_index()
  .nlargest(8, "n")
  .sort_values("m"))

# PLOT
fig, ax = plt.subplots(figsize
    =(6,5))
ax.barh(summary["Category"],
    summary["m"],
    color="#2E7D32", height=0.5,
    alpha=0.6, xerr=summary["se"],
    capsize=3, ecolor="#333")
for i, (m, s) in enumerate(zip(
    summary["m"], summary["se"])):
    ax.text(m+0.01, i, f"{m:.2f}"
```



pandas method chaining mirrors R's pipe. The chain goes from `read_csv` through `groupby().agg()` to a sorted summary, then plots with matplotlib.

Common Data Problems

Six Problems and Their Fixes

Six Common Data Problems and Their R/Python Fixes

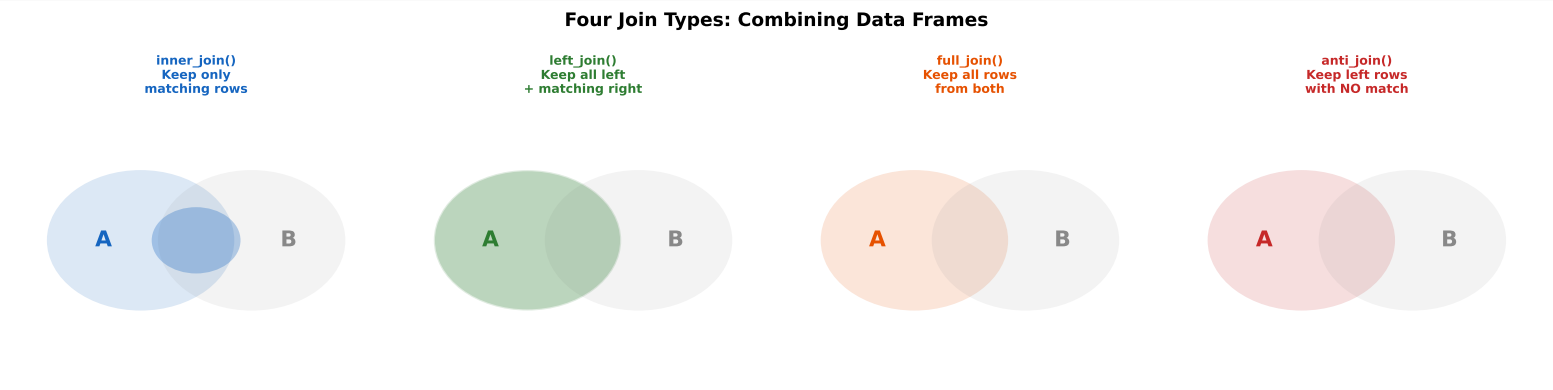
Problem	R Fix	Python Fix
Missing values (NA)	`drop_na() / replace_na()`	`dropna() / fillna()`
Wrong column type	`as.numeric() / as.Date()`	`pd.to_numeric() / pd.to_datetime()`
Inconsistent categories	`fct_recode() / str_to_lower()`	`str.lower() / replace()`
Wide format	`pivot_longer()`	`melt() / pd.wide_to_long()`
Duplicated rows	`distinct()`	`drop_duplicates()`
Outliers	`filter(between(x, Q1-1.5*IQR, Q3+1.5*IQR))`	`query() with quantile()`

Cleaning the Google Play Store Data

```r
# R: complete cleaning pipeline
apps_clean <- read_csv("googleplaystore.csv") |>
  # 1. Remove duplicates
  distinct(App, .keep_all = TRUE) |>
  # 2. Fix types
  mutate(
    Reviews = as.numeric(Reviews),
    Installs = as.numeric(str_remove_all(Installs, "[+,]")),
    Price = as.numeric(str_remove(Price, "\\$")),
    Last_Updated = mdy(`Last Updated`)
  ) |>
  # 3. Filter valid rows
  filter(Type %in% c("Free", "Paid"), !is.na(Rating)) |>
  # 4. Fix inconsistent categories
  mutate(Category = str_to_title(str_replace_all(Category, "_", " ")
      )) |>
  # 5. Remove outliers
  filter(between(Rating, 1, 5))
```

Joining Data

Four Join Types



Joins in R vs Python

R (dplyr)

```r
# Inner join
df <- apps |>
  inner_join(categories_meta,
    by = "Category")

# Left join (keep all apps)
df <- apps |>
  left_join(country_codes,
    by = c("Country" = "code"))

# Anti join (find unmatched)
missing <- apps |>
  anti_join(categories_meta,
    by = "Category")
```

Python (pandas)

```python
# Inner join (merge)
df = apps.merge(
    categories_meta,
    on="Category",
    how="inner")

# Left join
df = apps.merge(
    country_codes,
    left_on="Country",
    right_on="code",
    how="left")

# Anti join (no direct method)
missing = apps[~apps["Category"]
    .isin(categories_meta
          ["Category"])]
```

Synthesis

Wrangle-to-Viz Checklist

Before plotting, verify:

1. **Tidy?** Every variable is a column; every observation a row.
2. **Types correct?** Numbers are numeric, dates are dates, categories are factors.
3. **Missing values?** Handled by `drop_na()` or `replace_na()`.
4. **Duplicates?** Removed by `distinct()`.
5. **Reshaped?** Wide data pivoted to long if needed for `aes(color = variable)`.
6. **Aggregated?** `group_by() + summarise()` computed before plotting summaries.
7. **Sorted?** Categories reordered by value with `fct_reorder()`.

Key Takeaways

1. **Tidy data** (one variable per column, one observation per row) is the required input format for ggplot2 and seaborn.
2. **pivot_longer()** / `melt()` converts wide to long; **pivot_wider()** / `pivot_table()` does the reverse.
3. Six core verbs — **filter, select, mutate, arrange, group_by, summarise** — handle 90% of wrangling tasks.
4. The **pipe** (|> in R, method chaining in pandas) connects wrangling to plotting in one continuous flow.
5. **Always clean before plotting**: fix types, handle NAs, remove duplicates, reshape if needed.
6. **Joins** combine multiple data sources; use `left_join()` by default to preserve all primary records.

Essential Readings

- Wickham, H. (2014). "Tidy Data." *Journal of Statistical Software*, 59(10), 1–23.
- Wickham, H. & Grolemund, G. (2023). *R for Data Science*, 2nd ed., Chapters 3–7 (Data Transformation, Tidy Data, Joins).
- McKinney, W. (2022). *Python for Data Analysis*, 3rd ed., Chapters 7–8 (Data Wrangling, Joining).

Next Module: Comparative Lab — Grammar in Practice (W02-M10)